APIs

QSS20: Modern Statistical Computing

Where are we heading?

Class recaps

- Next week: Supervised machine learning

Recap of text-as-data

Natural language processing (NLP): Processing text for textual information.

Supervised methods: Humans imparting knowledge to computers *a priori*

Unsupervised methods: Have computers discover patterns for you

Recap of text-as-data

Natural language processing (NLP): Processing text for textual information.

Supervised methods: Humans imparting knowledge to computers *a priori*

Unsupervised methods: Have computers discover patterns for you

Sentiment analysis: Scoring words by their tone– positive, negative, or neutral.

Topic Modeling: Way to extract topics by feeding algorithms, such as Latent Dirichlet Allocation (LDA), documents.

Distrib. of topics over documents, words over topics.

Key concepts

Tokens: Individual units of language (i.e. split by space)

Parts of speech (POS) tagging

Overlaying tokens with POS, such as adjectives and proper nouns.

Named entity recognition

Identify specific names and places

Gensim is your friend

```
## Step 1: tokenize documents and store in list
text_raw_tokens = [wordpunct_tokenize(one_text)
                   for one_text in ab_small.name_lower]
## Step 2: use gensim create dictionary — gets all unique
    words across documents
text_raw_dict = corpora.Dictionary(text_raw_tokens)
## Step 3: filter out very rare and very common words
   from dictionary; feeding it n docs as lower and upper
    bounds
text_raw_dict.filter_extremes(no_below = lower_bound,
                              no_above = upper_bound)
## Step 4: map words in dictionary to words in each
   document in the corpus
corpus_fromdict = [text_raw_dict.doc2bow(one_text)
                   for one_text in text_raw_tokens]
```

Gensim is your friend

```
## Step 5: estimate a model by feeding it: (1) the corpus
## mapped to the dictionary, (2) the dictionary itself,
## (3) number of topics (in notation, k),
## and assorted other arguments
ldamod = gensim.models.ldamodel.LdaModel(corpus_fromdict,
                                         num_topics = 5,
                                         id2word=text_raw_dict,
                                         passes=6,
                                         alpha = 'auto',
                                         per_word_topics = True)
```

APIs

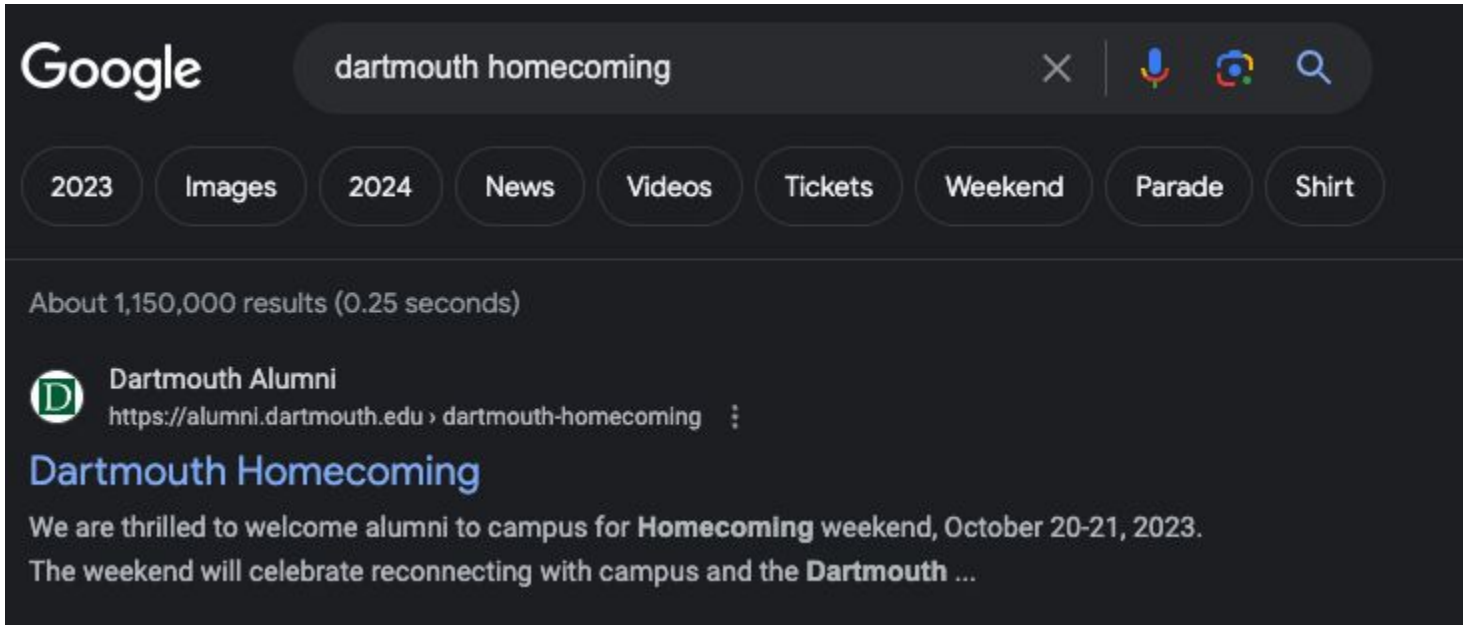
APIs that don’t need credentials

Application Programming Interface (APIs)

1. Construct a query that tells the API what data we want
2. Use request.get(query) to call the API
3. Examine the response–did the API return something (useful?)
4. If there is a response, extract the content

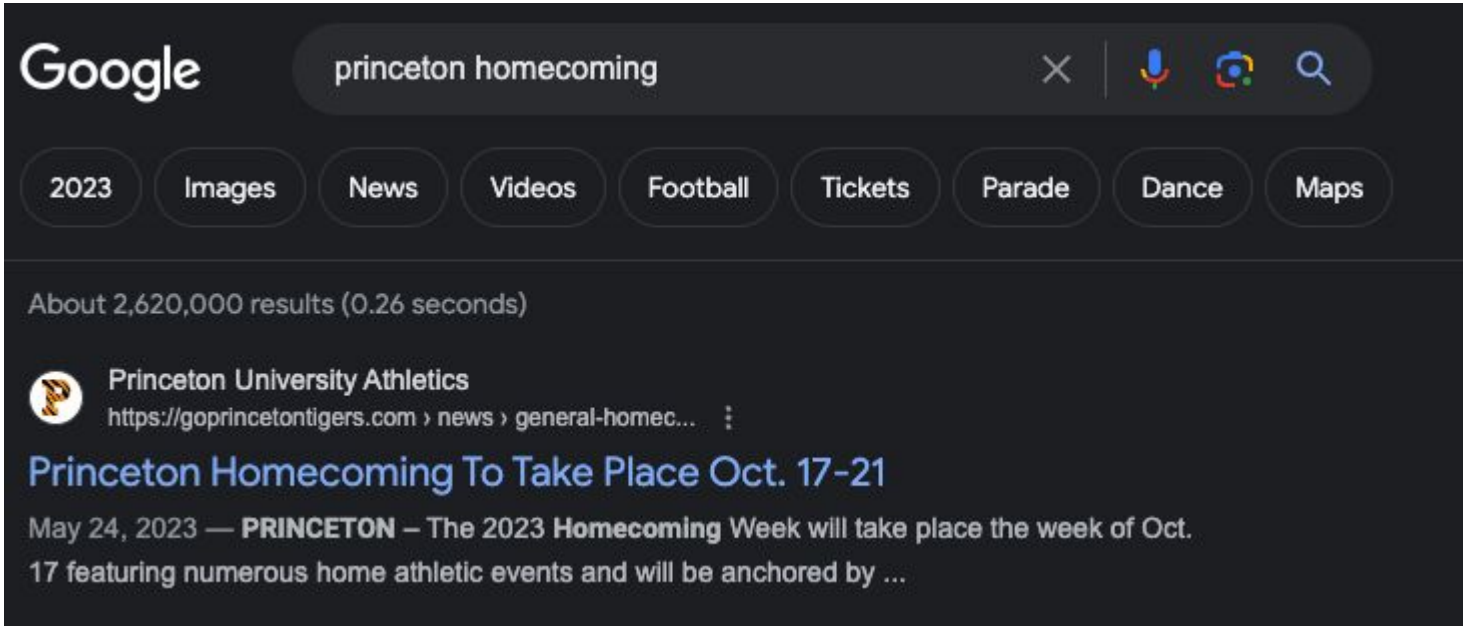
Familiar example: Google Searches

https://www.google.com/search?q=dartmouth+homecoming



Familiar example: Google Searches

https://www.google.com/search?q=princeton+homecoming



National Report Card

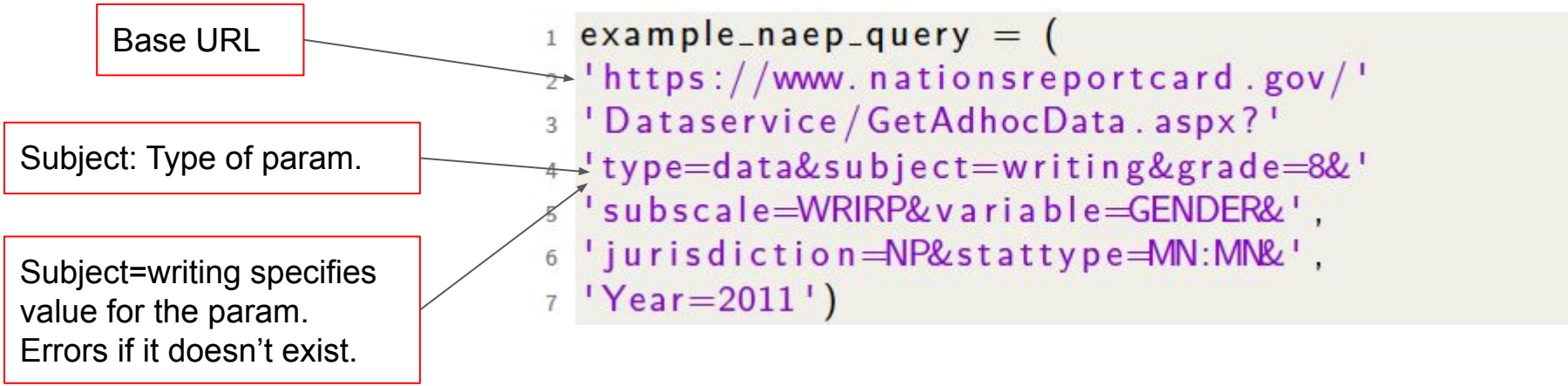
- NAEP provides State, county, and district level grade aggregates.

Step 1: Construct a query

Generic Example:

https://baseurl.com/q=something&date=somethingelse

NAEP example (you can use '( )' to split across lines



Steps 2-4: Call the API, example response, extract content

Use requests to call the API

Get JSON of the resp.

Extract content using a dictionary.

```
## use requests.get to call the API
naep_resp = requests.get(example_naep_query)

## we got usable response, so get json of status and
    result
naep_resp_j = naep_resp.json()

## extract contents in `result' key
## and convert to dataframe
naep_resp_d = pd.DataFrame(naep_resp_j['result'])
```

Activity 1: Practice pulling data from NAEP

Notebook:
https://github.com/herbertfreeze/qss20-x26/blob/main/activities/06_apis_blank.ipynb

- Example of query without errors
- Example of query with errors
- Combining API’s with functions to do multiple calls.

APIs that need credentials

Application Programming Interface:

1. Acquire Credentials (usually a key)
2. Construct a query that tells the API what data we want
3. Use request.get(query) to call the API

APIs that need credentials

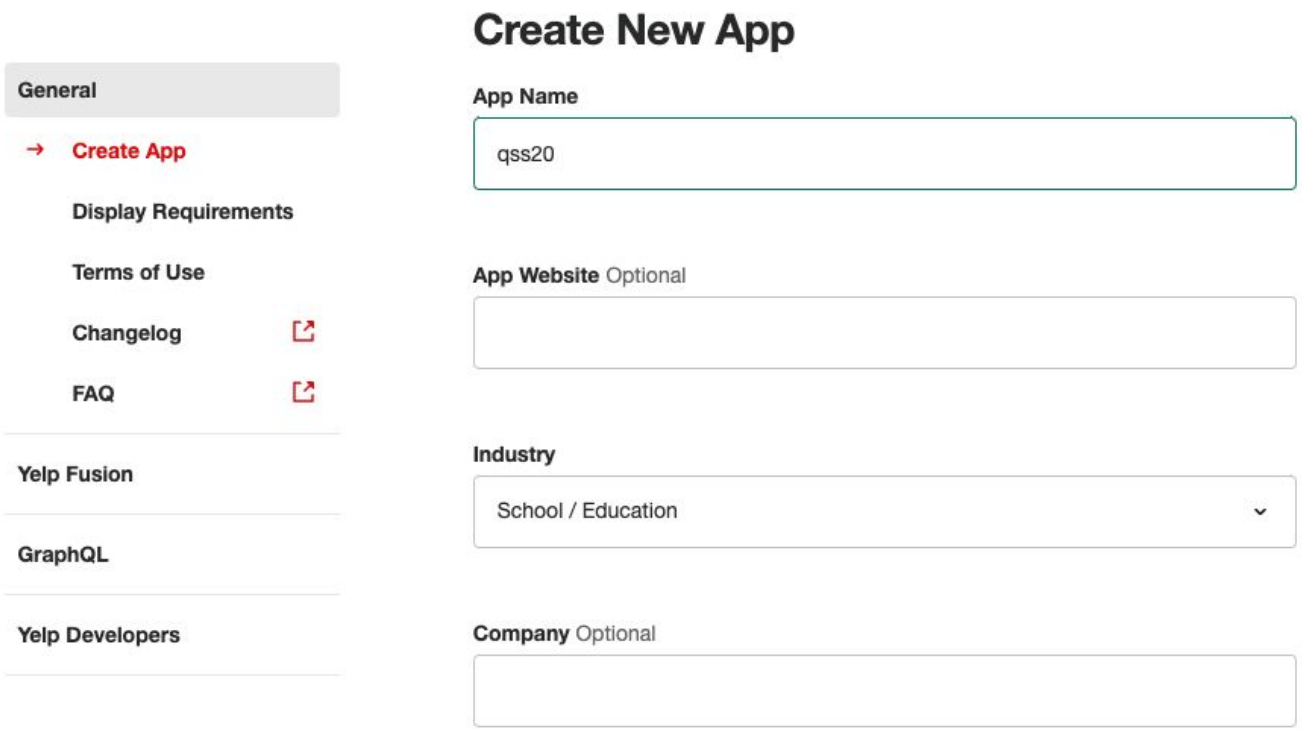
Application Programming Interface:

1. Acquire Credentials (usually a key)
2. Construct a query that tells the API what data we want
3. Use request.get(query) to call the API
4. Examine the response–did the API return something (useful?)
5. If there is a response, extract the content

Step 1: Acquire credentials

- Depends on the platform
- Examples:
  - Google Developer Console:
    Google Maps, YouTube, etc
    https://console.cloud.google.com
  - Yelp:
    https://www.yelp.com/developers/v3/manage_app

Step 1: Create Your Account



Step 1: Create Your Account

Use Dartmouth Public and don’t “login with Gmail”

Step 2: Construct a query

We follow the same process before, but focus on the **Yelp Fusion API.**

We will use the Business Search.

https://docs.developer.yelp.com/docs/fusion-intro

Name	Path	Description
Business Search	/businesses/search	Search for businesses by keyword, category, location, price level, etc.
Phone Search	/businesses/search/phone	Search for businesses by phone number.
Transaction Search	/transactions/{transaction_type}/search	Search for businesses which support food delivery transactions.
Business Details	/businesses/{id}	Get rich business data, such as name, address, phone number, photos, Yelp rating, price levels and hours of operation.

Step 2: Construct a Query

```
## defining inputs
base_url = "https://api.yelp.com/v3/businesses/search?"
my_name = "restaurants"
my_location = "Hanover,NH,03755"

## combining them into a query
yelp_genquery = (
    f'{base_url}'
    'term={my_name}'
    '&location={my_location}')
```

Step 3: Authenticate and call the API

Feed dictionary into the request's header parameter

```
## construct my http header dict
header = {'Authorization': f'Bearer {API_KEY}'}

## call the API
yelp_genresp = requests.get(yelp_genquery, headers =
    header)
```

Step 3: Authenticate and call the API

What is a successful call? <Response [200]>

What is an unsuccessful call? Let's try Hanover, WY, 09999

<Response [400]>

```
{'error': {'code': 'LOCATION_NOT_FOUND',
  'description': 'Could not execute search, try specifying a more exact location.'}}
```

Step 4: Extract results in a meaningful way

JSONs are nested dictionaries! Strategy: Save things in lists then make df.


```
{'id': '8ybF6YyRldtZmU9jil4xlg',
 'alias': 'mollys-restaurant-and-bar-hanover',
 'name': "Molly's Restaurant & Bar",
 'image_url': 'https://s3-media2.fl.yelpcdn.com/bphoto/1YkJFic4Czt9b2FsZyOrwQ/o.jpg',
 'is_closed': False,
 'url': 'https://www.yelp.com/biz/mollys-restaurant-and-bar-hanover?adjust_creative=A
gn=yelp_api_v3&utm_medium=api_v3_business_search&utm_source=ABQTB3e9fTiSiyqs0c-3Bg',
 'review_count': 403,
 'categories': [{'alias': 'tradamerican', 'title': 'American (Traditional)'},
  {'alias': 'burgers', 'title': 'Burgers'},
  {'alias': 'pizza', 'title': 'Pizza'}],
 'rating': 4.0,
 'coordinates': {'latitude': 43.701144, 'longitude': -72.2894249},
 'transactions': ['delivery'],
 'price': '$$',
 'location': {'address1': '43 South Main St',
  'address2': '',
  'address3': '',
  'city': 'Hanover',
  'zip_code': '03755',
  'country': 'US',
  'state': 'NH',
  'display_address': ['43 South Main St', 'Hanover, NH 03755']},
```

Casting all in “businesses” into a Dataframe

```
yelp_gendf = pd.DataFrame(yelp_genjson['businesses'])
```

Only retain columns that are strings

```
def clean_yelp_json(one_biz):

    ## restrict to str cols
    d_str = {key:value for key, value in one_biz.items(
    if type(value) == str}

    df_str = pd.DataFrame(d_str, index = [d_str['id']])
    return(df_str)

yelp_stronly = [clean_yelp_json(one_b)
        for one_b in yelp_genjson['businesses']]
yelp_stronly_df = pd.concat(yelp_stronly)
```

Activity 2: Practice with Yelp

- Run business search query for your hometown by constructing a query similar to yelp_genquery
- Take an id from “yelp_stronly.id” and pull reviews for that business, using documentation from this link: https://docs.developer.yelp.com/reference/v3_business_reviews
- Challenge: Generalize the previous step by writing a function:
  1. Take a list of ids as an input
  2. Call the reviews of the API
  3. Return the results
  4. Turn it into a dataframe